

An open virtual testbed for industrial control system security research

Bradley Reaves · Thomas Morris

Published online: 22 April 2012
© Springer-Verlag 2012

Abstract Industrial control system security has been a topic of scrutiny and research for several years, and many security issues are well known. However, research efforts are impeded by a lack of an open virtual industrial control system testbed for security research. This paper describes a virtual testbed framework using Python to create discrete testbed components including virtual devices and process simulators. The virtual testbed is designed such that the testbeds are inter-operable with real industrial control system devices and such that the virtual testbeds can provide comparable industrial control system network behavior to a laboratory testbed. Two virtual testbeds modeled upon actual laboratory testbeds have been developed and have been shown to be inter-operable with real industrial control system equipment and vulnerable to attacks in the same manner as a real system. Additionally, these testbeds have been quantitatively shown to produce traffic close to laboratory systems.

Keywords Virtual testbed · Industrial control system · SCADA · Cybersecurity

1 Introduction

Modern society has become dependent on critical infrastructure including water treatment, electricity generation/distribution, petroleum refining, and chemical processing. Failure of these systems can lead to significant financial and property

losses and physical harm to the citizenry. Industrial control systems, also known as supervisory control and data acquisition (SCADA) systems, manage complex and potentially dangerous processes by collecting data from sensors connected to a physical process and remotely controlling the process via electronic control of actuators, switches, and valves.

Currently, many researchers in the industrial control system domain have access to one kind of testbeds that are either constructed with real control system hardware and software or are virtualized environments modeling a single type of system. Researchers often identify vulnerabilities of their testbed, develop exploits against those vulnerabilities, and then capture normal and attack traffic using their private testbed. These private data sets are then used to validate new intrusion detection system research results. There are two major flaws in this private testbed approach. First, private data sets and testbeds do not allow repeating of experiments by third parties. Anomaly based intrusion detection systems use classifiers, which must be trained with pre-marked data sets. Problems with a data set may cause the classifier to rely on fields in data that are not truly related to an attack. This has happened with intrusion detection system research for enterprise networks [1]. Second, researchers may also tailor their intrusion detection system to their specific testbed. This can have the effect of potentially misleading intrusion detection system performance results. Third, as researchers continually publish intrusion detection system results for industrial control systems, there should be a way to repeat research by 3rd parties to verify results, and there should be a way to compare the performance of different published intrusion detection systems.

An important milestone in intrusion detection system development was the DARPA Intrusion Detection System Evaluations [2–4], which generated the DARPA data sets. This project simulated a medium-sized US Air Force local

B. Reaves
Georgia Institute of Technology, Atlanta, GA, USA
e-mail: brad.reaves@gatech.edu

T. Morris (✉)
Mississippi State University, Mississippi State, MS, USA
e-mail: morris@ece.msstate.edu

area network using a number of hosts running Expect scripts to simulate user activity (mail, FTP, HTTP, telnet). In the simulated traffic, a number of known network attacks were performed. The captured data (9 weeks' worth) were condensed into the KDD99 Cup data set [5], which provided researchers with a CSV file with connection information for the entire data set; the KDD data set was useful because it tagged which connections were malicious, making it possible to train and validate intrusion detection systems that use machine learning techniques. The DARPA data sets, the first of their kind, did have a number of flaws, which were only apparent in retrospect [1,6]. One major issue was that it had not been shown that the DARPA network traffic, being simulated, actually resembled true network traffic. Another issue was that all packets had four possible values for the IP TTL field, benign packets could have one of two TTL values, and malicious packets used one of two others. These flaws have led many researchers away from use of the DARPA data sets to validate intrusion detection research. This shows the flaw of simply releasing data sets. If a problem is found with a data set, it must be abandoned and replaced. In the case of the DARPA data sets, there is no way to go back and recapture data sets with the flaws removed.

Alternatively, open and revision-controlled virtual testbeds that model industrial control systems can be distributed to intrusion detection researchers. Researchers can use such a testbed to implement exploits of known vulnerabilities, capture network traffic from the control system under normal and attack conditions, validate intrusion detection systems within the virtual testbed, and finally, share comparable repeatable research results with other researchers in the field.

At present, it is difficult to quantitatively compare different industrial control system intrusion detection system approaches to determine which approaches are more promising and effective. Currently, there is no "standard" test scenario for industrial control system security solutions.

This paper describes an open virtual testbed platform developed to facilitate industrial control system intrusion detection system research while meeting the following properties. First, the open virtual testbed platform reduces duplication of effort as research groups do not have to create their own testbeds. An open platform enables researchers to update existing virtual testbeds or add new testbeds modeling new control systems. This results in a higher quality testbed for all researchers with less average effort for researchers. Second, the open virtual testbed platform provides a common ground for research enabling research groups to share code and enabling published results to be duplicated and compared. Third, the open virtual testbed platform may be used to generate captures of normal system network traffic and captures of attack or anomalous traffic for IDS researchers. These captures may be distributed with or independently of the testbed itself. This is a benefit to IDS researchers who

prefer to work with captures instead of live testbeds. If the system is open, distributed captures may be recreated to correct unexpected biases. This reduces the possibility of capture obsolescence. Fourth, the open virtual testbed platform opens the ICS security research area to more groups. Presently, industrial control system research requires substantial investment. With an open testbed, researchers can explore the area without having to purchase large laboratory setups. Additionally, amateur researchers and students can use the testbed without the requiring the financial backing of a large organization. Finally, while this paper later describes verification of the open virtual testbed platform, an open testbed can be audited and verified by anyone, because problems may be identified easier (according to the adage "Many eyes make bugs shallow") and corrected by any inclined researcher.

The following chapters detail the contribution of this work. First, a section on related works provides a summary of the state of intrusion detection system research for industrial control systems and information-related testbed projects. Next, an overview of the virtual testbed design goals, components, use cases, and the two implemented systems is provided. Next, the design of the virtual industrial control system devices for the testbed is discussed in detail. Next, the verification methodology, verification results, and use of the testbed are discussed. Finally, conclusions and future work are offered.

2 Related work

2.1 Industrial control system virtual testbeds

There are two principal approaches to creating a testbed: first, building laboratory-scale industrial control system with real equipment, or second, implementing virtual testbeds. Some researchers use small, laboratory-scale processes controlled by industrial control systems consisting of a few commercial or custom devices. Other researchers use virtual testbeds; these consist of simulated devices and may include a simulated process as well. A third option is a hybrid laboratory-scale and simulated system [7].

Laboratory-scale systems have a number of advantages compared to virtual systems. First, the data will reflect realistic measurement variations that would be present in an actual process control system. Second, the communication patterns and latencies will be entirely accurate and not vulnerable to inaccuracies in simulated variables like OS scheduling load. Third, industrial control system devices are individually vulnerable to many attacks that may not affect all systems. These vulnerabilities may not be present in the specification of the system that the virtual testbed was designed to emulate; examples include protocol implementation bugs that cause the device to be vulnerable to Teardrop attacks, LAND

attacks, web application attacks, or buffer overflows. Other security issues like poorly protected or hard-coded default passwords to devices will also not be present. With laboratory-scale process control systems, captures of these attacks can be provided in addition to protocol-based attacks and normal background traffic. In spite of their benefits, laboratory-scale systems have a number of disadvantages. First, laboratory-scale systems are expensive to develop and can be difficult to maintain. In particular, industrial control system software can be brittle and not user-friendly, and laboratory-scale processes require maintenance to stay operational. Second, adding or changing features in a laboratory-scale system can be difficult. Third, the size of laboratory-scale systems is limited by the required space and funds, so by necessity, the systems will be smaller and less featured than a real industrial control system.

Virtual systems can be simpler to develop and have practically no maintenance costs. Doubling the size of a virtual system requires only development time, not a large purchase order. Furthermore, virtual system configurations can be backed up and recalled instantly, so changing or adding features to a virtual testbed after does not permanently alter the system. However, making changes or adding features to a virtual system is arguably easier than in a laboratory system. While the captures taken from either a laboratory or a virtual industrial control system may be distributed, a major advantage of a virtual system is that a virtual system can be distributed widely to many researchers. For example, researchers may use a virtual testbed provided by another group to place intrusion prevention systems in the testbed to test effectiveness; such distribution is impossible with laboratory testbeds. Being able to distribute the virtual testbed also means that any researcher can recreate traffic captures if necessary; such capture regeneration may be necessary if biases or problems are found with previously released data sets. This openness can help the testbed and data sets avoid obsolescence. Virtual testbeds are also more convenient to use than laboratory testbeds for developing and debugging industrial control system security projects because they are portable and require little setup for an experiment. While virtual testbeds provide a number of benefits, there are some disadvantages to their use. Certain attacks, especially attacks that rely on device implementation errors, may not work against virtual testbeds. Also, virtual systems may not perfectly exhibit the same behavior as a real industrial control system.

In spite of the advantages of a laboratory testbed, a virtual testbed that is open and freely available will solve the research problems described in the introduction. First, an open testbed will reduce duplication of effort as research groups do not have to all create their own testbeds; rather, if the open testbed does not fit a group's needs, it may be improved in less time than creation of a new testbed. This results in a higher quality testbed for all researchers with less effort. Addition-

ally, other researchers may contribute virtualized versions of their laboratory testbeds for all to use. By enabling the creation of more diverse testbeds, the first two problems of the previous section can be solved.

Second, an open testbed provides a common ground for research; even if researchers wish to use their existing testbeds, they can also test and distribute projects in the open testbed. The benefit of this is that research groups can share code, and published results can be duplicated and compared. This solves the fourth issue discussed in the previous section.

Third, this testbed may be used to generate captures of normal system network traffic and captures of attack or anomalous traffic for IDS researchers. These captures may be distributed with or independently of the testbed itself. This is a benefit to IDS researchers who prefer to work with captures instead of live testbeds. If the system is open, distributed captures may be recreated to correct for unexpected biases. This reduces the possibility of capture obsolescence.

Fourth, an open testbed opens the industrial control system security research area to more groups. Presently, industrial control system requires substantial investment. With an open testbed, researchers can explore the area without having to purchase large laboratory setups. Additionally, amateur researchers and students can use the testbed without having to have the backing of a large organization.

Finally, while this paper later describes verification of the virtual testbed, an open testbed can be audited and verified by anyone. Because problems may be identified easier (according to the adage "Many eyes make bugs shallow") and corrected by any inclined researcher, the third issue of the previous section is solved.

While the testbed can be used to address prior problems, additional features can extend its usefulness. First, interoperability of the virtual system with actual industrial control system equipment will allow for hybrid testbeds and extend the usefulness of the virtual testbed. Second, designing the testbed such that important characteristics like industrial control system protocols or communications interfaces can be changed will greatly enhance the ease of use of the system, especially compared with laboratory systems. Third, designing the testbed such that components can be replaced or extended easily will encourage use, reuse, and contribution to the testbed.

A number of universities and government agencies are developing or have developed testbeds for studying SCADA system attacks. These testbeds are outlined below; following that discussion, the industrial control system and Electric Grid testbeds at MSU are discussed in detail.

Giani et al. proposed, but did not implement, a SCADA testbed [7]. First, they defined reference architecture of a SCADA system. Next, the authors considered three types of testbed: single simulation (all in a simulation framework like MathWorks' Simulink), implementation-based (where real

SCADA devices are used), and a federation simulation (simulation is combined with real SCADA devices). To tackle the problem of device synchronization, the authors recommended the use of the US Department of Defense High Level Architecture system.

In a Master's thesis, David Bergman developed a simulated testbed for modeling Electric Grid SCADA systems [8]. A network simulator, RINSE, was used to model communications between simulated devices; these simulated devices include relays and data aggregators. PowerWorld software was used to simulate the Power Grid. This simulated testbed has the ability to be integrated with real hardware. This work will not be released to other researchers. Additionally, no verification was done to ensure that the simulated testbed traffic is similar to actual traffic. Furthermore, this testbed is aimed for electric grid control systems and not general industrial control system.

In [9], the authors take the approach of using a small "complex electromechanical device consisting of pipes, valves, sensors, pumps, etc" to model a power plant. Their system uses a number of different PLCs and field devices. They also use a DeltaV DCS system. They also included a small office intranet with interconnections with VLAN, VPNs, RADIUS, a DMZ, and "external network" system. The authors have used this system to demonstrate four different attack scenarios.

2.2 MSU SCADA security laboratory

Mississippi State University's laboratory-scale process control system cybersecurity testbed [10] uses process control system equipment to control and monitor small, laboratory-scale processes. MSU has two types of testbed: five laboratory-scale industrial control system and a laboratory-scale electrical substation (in which power flows are simulated). The industrial control systems include an oil pipeline system, an oil storage tank, water tower, industrial air blower, manufacturing conveyor, and rolled sheet metal plant. The rolled sheet metal plant and electric substation simulators use Allen-Bradley PLCs controlling their systems using the Ethernet/IP protocol. The remaining systems are controlled by single Control Microsystems PLCs communicating over MODBUS wirelessly to a single master PLC unit, which is in turn connected to an HMI. The electric substation testbed is described further in detail in [11].

Two of the industrial control systems—pipeline and the oil storage tank—are discussed in later chapters, so a more thorough treatment of them will be given here. The oil pipeline model system consists of a pipe with a pump, an electrically controlled release valve, and a pressure meter, all connected to the controlling slave PLC. Air is used in place of oil for safety and simplicity. The system has three modes of operation: off, manual, and auto. In off mode, the slave closes the

valve and turns off the pump. In manual mode, the pump and valve status are set by the master PLC based on input from the HMI. In automatic mode, a PID loop is used to maintain the pressure at a setpoint specified by the master PLC based on input from the HMI. Two modes may be selected: pump mode, where the valve is always open and the pump is modulated on and off to control the pressure, and solenoid mode where the pump is always on and the valve is opened or closed as necessary to control the pressure. The ground tank system uses water in place of oil for safety reasons and consists of a plastic water tank, a reservoir, and a pressure gauge and a pump connected to the slave PLC. The reservoir holds water not in the tank, and the pump moves water from the reservoir to the ground tank; a pipe drains water continuously from the tank to the reservoir. Like the pipeline, the ground tank has auto, off, and manual modes. The off and manual modes work similar to the pipeline system. The auto mode has a low and high setpoint; the pump is turned on when the water level is below the low setpoint and turned off when the water level is above the high setpoint.

3 Overview of the virtual testbed

The virtual testbed should meet two sets of goals: functional goals, which govern testbed capabilities, and characteristic goals, which govern qualities that the testbed should have.

The main functional goal of the testbed is to be able to emulate realistic serial and TCP/IP industrial control system protocol communications. These captures must be realistic in protocol features, industrial control system behavior, and include industrial control system data. Ideally, captures from a real system would be indistinguishable from captures from the virtual testbed; this will likely never be the case, though, so it is important that amount of similarity of traffic be known quantitatively. In addition to the industrial control system traffic, it is important that the process under virtual industrial control system control can also be shown to be similar to actual physical processes.

Captures of the network traffic, to be effective for research purposes, must be logged reliably; logging should be done with a minimum effect on the industrial control system traffic itself and with sufficient precision and accuracy to fully describe the behavior of the system. In addition to exhibiting realistic traffic, the testbed should also be able to interface with actual industrial control system equipment. This helps to ensure that the testbed has behavior compatible with real industrial control systems and facilitates the use of the system as a "backend" for research on equipment vulnerabilities. This requirement dictates that the virtual testbed must be capable of soft real-time operation.

In order for the benefits of an open system to materialize, the virtual testbed system must be flexible and easily

extended. Each component should stand independent of other components; no individual component must be essential to operation. This aids in the objective that the virtual testbed integrate with existing systems; it also allows researchers to adapt other projects to work alongside this virtual testbed. Examples include the ability to include PMU and PDC instances from the OpenPDC project or to integrate other software virtual testbeds.

This principle of “interchangeable parts” dictates that components must have well-defined, modular interfaces. Application of this principle mandates that data sharing between components should be standardized internally, if not standards-based. Another implication is that it should be very simple to convert systems using one industrial control system protocol to another; this will facilitate ease of use by researchers and lead to the easier creation of security techniques and systems that function for many industrial control system protocols—not the single protocol supported by a testbed instance. For this reason, much of the specification of system behavior and characteristics should be maintained in text-based configuration files. Such files also promote usability and the ability to troubleshoot errors; this is especially true in comparison to the configuration methods for actual industrial control systems that rely on buggy GUIs for configuration.

3.1 Components

The design of the virtual testbed is broken up into discrete components. The main components are the process simulator, the virtual device, actual devices, configuration files, and data loggers to capture and store network traffic and process state. Figure 1 shows how these components form a complete testbed. The process simulator models the mechanics of a physical process. The process simulator may be implemented as a Python class or commercial process simulators may be used with an interface class. At the center of the diagram

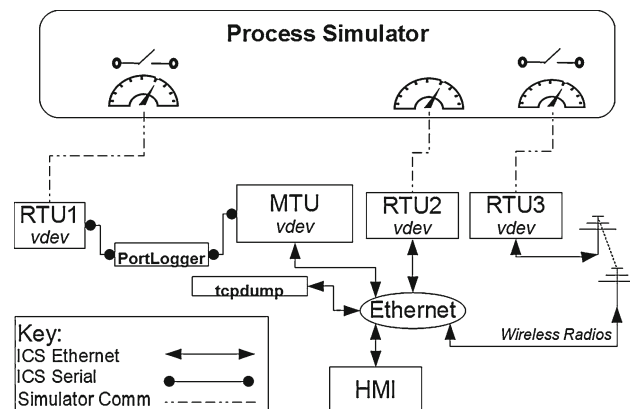


Fig. 1 Open virtual testbed architecture

are virtual devices taking the places of master terminal units and remote terminal units in an industrial control system. Virtual devices include control logic implemented in Python to mimic internal master or remote terminal logic functions (such as ladder logic). Additionally, virtual devices implement points which model internal memory of the remote or master terminal unit. The virtual devices and process simulator communicate via a separate simulator communication channel. Figure 1 also includes a set of wireless radios and a human-machine interface (HMI) computer. The wireless radios and HMI are examples of actual device. The virtual testbed architecture supports interface with virtual and actual devices. Configuration files are used to describe connections between testbed components and to set communication protocol and speed for each interface. Finally, data loggers are available to capture and store process information and network traffic. The PortLogger shown in Fig. 1 captures serial port network traffic, and tcpdump is used to capture Ethernet traffic.

All components were written in the Python programming language. Traditional network simulation toolkits, like NS-2, NS3, and OMNet++, were not used for several reasons. First, traditional network simulation toolkits feature a heavy burden in both design complexity and learning curve. For testbeds based on network simulation toolkits, not only does a new researcher in industrial control system security have to learn about industrial control system, but he must also learn the toolkit. By eschewing the complexity of traditional toolkits, this project hopes to increase user-friendliness while not sacrificing performance relative to the goals discussed in the previous section. Second, traditional network simulation toolkits focus on providing detailed modeling of system behavior for performance optimization; this includes fine-grained modeling of queuing effects and myriad processing delays. By contrast, this project is concerned only with the level of detail required by industrial control system security researchers—not industrial control system network designers. Third, traditional network simulation toolkits focus primarily on TCP/IP and wireless networks, and many existing testbeds are also focused almost exclusively on TCP/IP industrial control system networks. This project aims to provide an emphasis on serial-communications-based systems while not neglecting TCP/IP industrial control system networks. Finally, traditional network simulation toolkits are designed to have the flexibility to model large network behaviors; by contrast, as most industrial control systems number in merely tens of nodes, the complexity of being able to model large networks is not needed.

3.1.1 Process simulator

The first component of the virtual testbed is the process simulator. The process simulator is meant to simulate and

model the mechanics of a physical process being controlled by an industrial control system. For example, if a testbed was designed to model an oil storage tank, a process simulator would be necessary to manage the physics of filling the tank, including enabling the pumps and drain valves. Process simulators must simulate not only the physics of the system, but also be responsive to control inputs from the industrial control system (virtual or real), just as in a real industrial control system. For industrial control system security research, it is vital to be able to model the effects of attacks and counter-measures not just on the industrial control system equipment but on the actual process under control.

The virtual testbed process simulator is designed to communicate directly with some or all of the devices in the virtual testbed; this communication occurs on a “back channel” separate from the industrial control system communication. The communications consist of measurements from the simulator and inputs to the simulation from the devices. The data that come from the simulator are meant to mimic the analog and digital inputs found in PLCs and other industrial control system devices used for temperature, pressure, flow, and other gauges. The data delivered to the simulator are meant to emulate the analog and digital outputs on industrial control system devices that are used to control actuators like motors, pumps, and valves.

The process simulator consists of four components: a simulator module, a communication interface, an update queue, and configuration files. The simulator module performs process simulation and stores the current state of the physical process. The communication interface receives periodic process control changes and requests for process measurement updates from virtual devices (VDEV). Requests from the VDEVs are queued by the communication interface and applied in the order received by the update queue. Configuration files are used to set initial conditions for the physical process. The interface between VDEVs and the process simulator is separate from communication channels between VDEVs. The process simulator to VDEVs interface models analog and digital input and outputs typically found on a remote terminal unit or intelligent electronic device. The process simulator architecture is shown in Fig. 2.

Process simulations are described in Python classes. At start-up, the simulator module reads the configuration file to configure process variables, such as flow rates, motor speeds, etc., and sets the initial simulation state.

Simulation is discrete time based. The simulator interface accepts and provides updates to and from the simulator about relevant inputs and outputs of the device. Inputs and outputs are defined in a configuration file by name. Name–value pairs are transmitted to and from the process simulator. This simulates the connection of analog and digital input and outputs between the remote terminal unit (simulated by a VDEV) and process measurement devices and process actuators.

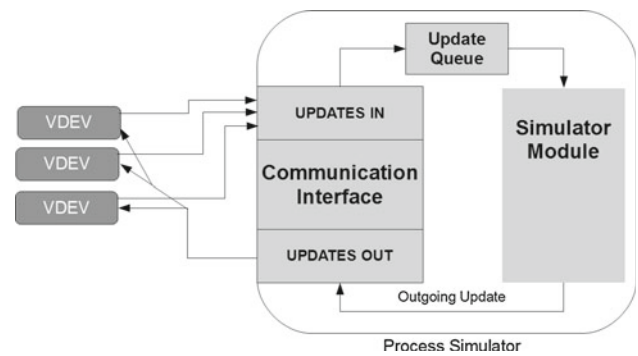


Fig. 2 Process simulator architecture

3.1.2 Virtual devices

Distributed control systems contain specialized computational devices to implement control schemes and communicate within the control system hierarchy. These control devices include remote terminal units (RTU), master terminal units (MTU), programmable logic controllers (PLC), intelligent electronic devices (IED), repeaters, and programmable automation controllers (PAC). Henceforth, this class of controller will be referred to as a RTU in reference to the entire class of devices. Virtual devices (VDEV) model RTU in the open virtual testbed.

VDEVs are the primary components of the virtual testbed. VDEVs include communication interface(s) to communicate with control system computers, actual RTU, or other VDEV. Currently, the open virtual testbed environment supports MODBUS/TCP and MODBUS/RTU protocols. VDEV also includes a communication interface to communicate with process simulators. This communication interface is implemented as a UDP/IP link on a local area network, which is separate from any MODBUS/TCP local area network used for the control system. VDEVs implement control logic, which emulates control logic found in RTU. VDEVs make use of abstract objects termed “points” that allow for the myriad memory organization and addressing schemes used in industrial control system devices.

VDEVs are implemented as processes that can run stand-alone or in a virtual machine. Executing the VDEV within a virtual machine allows the VDEVs to leverage the independence of the virtual machine operating system’s network interface. Using standard network interfaces allows VDEVs to be interchangeable with actual control devices.

VDEV to VDEV/RTU/PC Communication: As mentioned above, VDEVs are implemented within a virtual machine and use the operating systems network software stack for all communication layers except application layer.

For MODBUS/TCP, the physical layer may be a virtual LAN implemented by the virtual machine software suite or an actual LAN. The network and transport layers are

implemented by the operating system. For MODBUS/RTU, the physical layer may be a virtual serial port connection implemented by the virtual machine software suite or an actual serial port connection. The application layer (MODBUS) is implemented within Python using the MODBUS-TK Python MODBUS library. Using the MODBUS-TK Python MODBUS library and the operating system network software stack for network and transport layer implementations allows virtual devices to communicate with actual devices.

For implementing the protocol interfaces in the virtual testbed, two Python modules were created: `ics_clients` and `ics_servers`; these modules are meant to contain implementations of protocol clients and servers, respectively. These implementations may wrap existing Python libraries, be created from Python bindings to existing code in other languages (such as C or Java), or be designed from scratch. Base client and server classes were written to outline the software interface of the clients and servers for new protocol implementations; actual protocol clients and servers can override the methods from these base classes. If uniform methods are used for client and server interfaces, protocols can be changed without having to change virtual device logic. Switching from one protocol to another can be done by instantiating a different industrial control system interface Client/server type, and the change will be invisible to the rest of the VDEV. For this reason, client and server base classes were written with certain attributes and methods that are common to all protocols. This allows control logic to be implemented in a protocol agnostic form and supports the ability to change communication protocol without need to rewrite control logic.

All protocol interfaces assume they will be connected to actual communications ports; this may be a network connection or a serial device. For testbed use cases where external connections to other VDEVs on other hosts or to actual devices are necessary, this assumption is helpful; if only internal connections to other VDEVs are required, virtual networks (enabled by virtual machines as VDEV hosts) or virtual serial ports can be used.

Other tasks often performed by an industrial control system device include acting as FTP or HTTP servers. This functionality is not provided natively in the VDEV. However, the functionality may be emulated by running an FTP or HTTP server alongside the VDEV in a virtual machine. Other services may also leverage Python's extensive libraries to implement a server if required.

VDEV to Process Simulator Communication: VDEVs include an interface to communicate with the process simulator. The process simulator interface accepts and provides updates to and from the simulator about relevant inputs and outputs of the device. The process simulator interface emulates common connections from RTU to measurement devices (inputs) and actuators (outputs). RTU typically

include digital inputs and outputs and links to analog to digital (ADC) and digital to analog (DAC) converters. The process simulator inputs and outputs to emulate these interfaces may be single bits, integers, and floating point numbers.

VDEV Control Logic: The control logic directly emulates the control and monitoring functions of an actual control device. Control logic is specified as a single Python function separate from the VDEV implementation. The control logic is called by a logic control thread; this thread also synchronizes process simulator communication. The same control logic function may be used for multiple slave devices to allow code reuse. Process control logic in actual industrial control systems consists of reading inputs, performing logical and numeric calculations, and then setting appropriate outputs. This behavior is modeled by the VDEV control logic. Reads and writes to points in device memory are handled with `get()` and `set()` functions. Control logic may consist of combinatorial logic, loops, conditional programming, and arithmetic.

Figure 3 shows example control logic for an on/off controller to safely turn on a water storage tank pump. The code checks the status of the control system mode (manual or automatic), reads the value of high-level and low-level water sensors, and the current intended pump state before turning the pump on.

The code in Fig. 3 does not directly address memory points. Instead points are referred to by name. Also, the control logic does not include protocol-specific commands. The `get()` and `set()` functions are implemented above the protocol layer to allow use of control logic with multiple communication protocols.

Figure 3 shows a nested if block to implement an on/off control scheme. More complicated control schemes such as proportional–integral–derivative (PID) control are also possible.

VDEV Points: Points are data objects stored by the VDEV, emulating actual device memory. Points provide a protocol agnostic interface and nomenclature for data objects. Points provide a memory abstraction by allowing control logic within the VDEV to refer to data objects by name

```
#determine if pump should be turned on
if (points['SystemInAuto'].get() and
    not points['HighLevelFloat'].get() and
    (points['LowLevelFloat'].get() or
     points['PumpRunCmd'].get() )):
    points['PumpRunCmd'].set(TRUE)
elif (points['SystemInMAN'].get() and
      points['MANPumpRunCmd'].get()):
    points['PumpRunCmd'].set(TRUE)
else:
    points['PumpRunCmd'].set(TRUE)
```

Fig. 3 VDEV control logic example

independent of the communication protocol or the memory assignment. This allows the communication protocol or the memory assignment to change without need to change the control logic or other code, which refers to points. Points may hold a value of any data type—integer, string, floating point, binary file object, or even a Python class.

3.1.3 Actual devices

Because virtual devices are implemented within virtual machines and use standard communication interfaces and validated network software stacks, actual industrial control system devices may be connected to a virtual testbed. This feature allows virtual testbed users to test new devices with a virtual testbed and capture data logs, which include interaction with new actual devices.

For example, a virtual testbed may interface with actual human machine interface (HMI) software. HMI software available in the MSU SCADA Security Laboratory communicates with the control system via MODBUS/RTU. The HMI can run in one virtual machine and communicate with a VDEV, which simulated a remote terminal unit.

A second example of using an actual device with a virtual testbed involves the use of industrial radios. Industrial radios available in the MSU SCADA Security Laboratory act as MODBUS/RTU repeaters. Actual industrial radios were placed in series between a virtual master terminal unit and remote terminal unit. Introduction of the radios did not alter simulation except to delay receipt of MODBUS/RTU packets transmitted between the master terminal unit and remote terminal unit, an expected result.

3.1.4 Configuration files

Both the process simulator and the VDEVs are highly modular, and system characteristics and behavior, apart from the process control logic and the process simulation, are described by a single configuration file. This file is text-based and can be hand-edited or machine-generated. The file contents include simulator configuration information and configurations for each VDEV in a system. Simulator configuration information includes simulator interface type, interface configuration information, and simulator variable initialization. VDEV configuration information includes the number and names of each VDEV in the system, the data objects to be stored (points), interface timeouts, communication protocol information, server and client protocol types and number, and industrial control system interface configurations (addresses, ports, et cetera).

The use of configuration files allows a virtual testbed to be expanded with copies of existing process simulators and

virtual devices. In this way, a virtual testbed can grow in scale unlike an actual testbed.

3.1.5 Data logging

A primary requirement of a virtual testbed is the generation of useful data sets to support research. Ideally traffic capture should not affect testbed operation.

MODBUS/TCP network traffic captures are possible with either tcpdump or Wireshark. These tools can be run in promiscuous mode to allow traffic capture without interfering with control system communications. However, this leads to network traffic logs in which client TCP port number must be used to distinguish between VDEVs. Hosting each VDEV in its own virtual machine allows assignment of unique IP address to each VDEV.

MODBUS/RTU data logging (or any serial port data logging) requires a special, PortLogger. PortLogger solves three problems. First, PortLogger allows creation of virtual serial ports to connect multiple VDEVs. Second, PortLogger provides the ability to connect multiple virtual ports to one or more physical serial ports. Third, PortLogger captures and stores all data transmitted by serial devices (virtual or actual).

PortLogger creates a pseudo-terminal master/slave pair for each virtual device requiring a serial port; the slave port is provided for the device (or a symbolic link is created from the slave port to a file that the device expects to read), while the master port is opened by the PortLogger. When a device sends a message, it is read by the PortLogger, echoed to the other devices, and logged. The PortLogger can also open a physical serial port and echo to and from it as if it were a pseudo-terminal. It is also possible to designate a single device as a “Master” to emulate a master-to-slaves multipoint RS-485 system. The PortLogger can also insert a delay between receiving a message and retransmitting it to emulate transmission delays.

When data logging, it is important not to modify or impede the flow of traffic. Logging techniques that place an active device between two devices to be measured, that is, “bump-in-the-wire” systems, cannot be used to obtain traffic captures that contain timing information that reflect traffic conditions when the system is not being logged. For actual systems, serial tap cables should be used; these are cables used to connect two serial devices that have additional connectors on each data line for a logger to listen to transmissions. For virtual testbed systems, PortLoggers are used; the problems seen with bump-in-the-wire logging are not present because the logging occurs before the transmission time delay. This is analogous to a tap cable. If PortLoggers are used to connect virtual devices to hardware serial ports, the same problems with bump-in-the-wire logging are seen.

4 Evaluation

Validation of the virtual testbed framework was accomplished in four phases. First, two actual industrial control systems from the MSU SCADA Security Laboratory were modeled as virtual testbeds. Second, the two virtual testbeds were integrated with actual control system devices to demonstrate that actual systems cannot distinguish between the virtual devices and actual devices. Third, a comparison of network traffic between actual and virtual testbeds was performed. Finally, a comparison of measurements from a process simulator and an actual process was performed.

4.1 Virtual testbed implementations

A gas pipeline and a water storage tank control system were modelled with virtual testbed implementations for validation of the virtual testbed platform. Descriptions of the actual systems are provided in the related works section.

Figure 4 shows the water storage tank and gas pipeline systems from the MSU SCADA Security Laboratory. Both systems implement a distributed control scheme. A Control Microsystems, Inc. SCADAPack LP PLC is connected to each physical process and acts as the remote terminal unit (RTU). The PLC includes ladder logic that monitors process measurements and executes control logic. The control logic is described in the form of ladder logic. A single MTU communicates with both RTU. The MTU includes supervisory control logic and acts as a repeater forwarding queries from a system HMI to the addressed RTU. The HMI periodically polls each RTU for process measurements. The HMI can also execute supervisory control over the physical processes by changing RTU set-points.

All communication for both systems is MODBUS/RTU. The virtual testbeds were also operated in MODBUS/TCP mode. This mode is not available on the actual systems.

4.2 Actual versus virtual network traffic comparison

Comparing network traffic captures is a non-trivial task when one considers the high number of features that may be considered. Terry Brugger [12] developed a methodology for comparing IP network captures for use in IDS testing. He created “similarity metrics,” which can produce a scalar value from 0 to 1 denoting how similar two captures are based on network and traffic characteristics; 1 is considered “identical,” while 0 is considered “not-identical.” Each metric measures a certain characteristic of the captures under comparison.

There are four types of metrics: scalar metrics, discrete metrics, ordered continuous metrics, and sorted continuous metrics. Scalar metrics, also termed “total number characteristics,” are metrics that consist of a single value like the number of hosts. Scalar metrics are defined by computing a single value (or score) for each of two captures. Similarity of the two captures with respect to these values is defined by Eq. 1 where x_1 and x_2 are the scores of the captures.

$$\text{similarity} = 1 - \frac{|x_1 - x_2|}{x_1 + x_2} \quad (1)$$

Discrete metrics are keyed integral metrics; that is, one integer score is calculated for a finite set of key values. The similarity for a discrete metric is defined by Eq. 2, where $x_{1,i}$ is the i th scalar metric in the first capture and so on. An example is a metric measuring number of bytes transmitted per host; the host is the key, and the byte count is the scalar metric.

$$\text{similarity} = 1 - \frac{1}{n} \sum_{i=0}^n \frac{|x_{1,i} - x_{2,i}|}{x_{1,i} + x_{2,i}} \quad (2)$$

Ordered continuous metrics are keyed non-integral, or real number, metrics. This type of metric was not used for the evaluation of the virtual testbed and will not be discussed further. Sorted continuous metrics are based on applying a function to a capture that generates a sequence of values; an example of such a metric is inter-arrival time between packets. Once



Fig. 4 Actual control systems from MSU lab. **a** Water storage tank, **b** gas pipeline

the two sequences are generated, their values are sorted, and the similarity is with Eq. 2 in the same manner as the discrete metrics. If one of the sequences is longer than the other, the longer sequence must be reduced to be of the same length as the shorter for the metric to be calculated correctly. The following algorithm was used to reduce the longer sequence size. First, for each element in the smaller list, calculate a “normalized” index by dividing the index of the element by the length of the shorter list. If the normalized index is an integer, append the value of the sequence at that index to the new sequence. If the normalized index is a real, append the average of the values of the two elements in the larger list closest to that index.

In addition to the metrics above, a sequence metric was developed. The sequence metric counts the number of occurrences of unique subsequences of length l within a larger set, S . The ordered list of subsequence counts is used as arguments to the discrete similarity described above to compute a similarity score. Industrial control system network traffic is generally deterministic. Systems read and write to the same registers in the same order at regular intervals. Sequence metrics were used to that this deterministic behavior is present in the virtual testbed and matches that of a real system. Simple sequence metrics were used to show that master requests are always followed by a response. Other sequence metrics were used to show that the order in which a master accesses a set of registers or communicates with a set of slaves is maintained by the virtual testbed.

A set of metrics were defined for MODBUS/RTU systems. The metrics were chosen to ensure that traffic features commonly used in model-based intrusion detection systems [13, 14] would be shown to be represented accurately in the virtual systems. The metrics used include capture characteristics, packet characteristics, and timing characteristics. The following metrics were defined:

Byte Throughput: Count of total data bytes in a capture divided by the time from the first packet to the last packet. Computed as a scalar metric. This metric was included as it is a very basic, but important, descriptive metric of network traffic.

Packet Throughput: Count of total number of packets in a capture divided by the time from the first packet to the last packet. Computed as a scalar metric. This metric was included as it is a very basic, but important, descriptive metric of network traffic.

Error Count: Count of number of packets is marked as MODBUS response errors (the high bit of the function code field is set). This is divided by the total number of packets to normalize comparisons between captures of different sizes and is computed as a scalar metric. Error Counts are measured and compared because a significant difference in errors indicates that two systems handle incoming packets differently (and not according to the specification).

Invalid CRC: Count of number of packets with bad checksums. This is divided by the total number of packets to normalize comparisons between captures of different sizes and is computed as a scalar metric. This metric is important as invalid CRCs may be indicators of attacks (especially radio jamming).

Function Code: Count of the number of packets with a given function code (from 1 to 127), calculated for each function code. This is a discrete metric. This is included because the function codes and their proportions define the behavior of the network—what data are being read and written.

Function Code Sequence: This is a sequence metric that examines runs of packet function codes. The length used is 2. This metric is used to verify that network traffic proceeds in the same order in both systems—a very basic, yet important feature.

ID Sequence: This is a sequence metric that examines runs of Slave IDs in MODBUS/RTU packets. The length used is 2. This metric is used to verify that network traffic proceeds, particularly with respect to slave polling order, in the same order in both systems — a very basic, yet important feature.

Packet Size: A list of all packet data lengths in the capture. This is a sorted continuous metric. This metric is included because different packet sizes would indicate that different amounts of data were being exchanged in one system rather than another.

Inter-arrival: Time A list of the amount of time that passes between two consecutive packets for all packets in the trace. This is computed as a sorted continuous metric. This metric acts as an aggregate of all timing information and ensures that traffic proceeds at the same rate in both systems. As an example, this is relevant to intrusion detection and attacks that rely on timing to inject packets.

Master-to-Master Inter-arrival Time: A list of the amount of time that passes between two consecutive packets sent by the Master device for all Master device packets in the trace. This is computed as a sorted continuous metric. This metric acts as a proxy to measure the time between requests, an important system feature.

Master-to-Slave Inter-arrival Time: A list of the amount of time between master requests and slave responses. This is computed as a sorted continuous metric. This metric measures, in effect, the amount of time it takes for a slave to process and respond to a request.

Similarity values for all metrics are presented for all the above metrics. Continuous and discrete metrics also include the average error, the maximum error, and the percent error. An error list is computed by subtracting corresponding elements of the simulated system metric list from the laboratory system metric list; lists with differing sizes are accounted for using the same technique as continuous metrics—choosing corresponding elements by normalized indexes. Average error is the arithmetic mean of the error list, and maximum

Table 1 Similarity metrics for the ground tank system (off mode)

Name	Similarity	Average error	Max. error	Percent error (%)
Byte throughput	0.98493	4.32641	4.32641	−2.969
Error count	1.00000	—	—	—
Function code count	0.99996	−0.00000	0.00032	0.000
Function code sequence	0.99964	—	—	—
ID sequence	0.99995	—	—	—
Inter-arrival Time	0.93533	0.00366	0.06616	3.046
Invalid CRC	1.00000	—	—	—
Master-to-aaster inter-arrival time	0.94912	0.00731	0.07220	3.038
Master-to-slave inter-arrival time	0.91356	0.00548	0.01950	14.420
Packet Size	0.99999	−0.00083	0.00000	−0.004
Packet throughput	0.98495	0.24693	0.24693	−2.966

error is the maximum value of the error list. Percent error is calculated as using Eq. 3 where $\text{avg}()$ is the arithmetic mean operation and S and L are the simulation metric list and the laboratory metric list, respectively.

$$\% \text{error} = \frac{\text{avg}(S) - \text{avg}(L)}{\text{avg}(L)} \quad (3)$$

Table 1 gives the similarity metrics for a comparison of the ground tank simulation system and the MSU laboratory-scale system. The capture taken from the MSU laboratory system was acquired by directly connecting the master and slave units with a tap cable and recording the traffic; the system was left switched in the off mode for 291 s, and 2,419 packets were captured. The capture taken from the virtual system was created using a PortLogger with baud rate emulation with the virtual system also in off mode; 3,173 packets were captured in 393 s. The same processes were repeated to obtain system captures of the systems in auto mode. The virtual system auto mode capture consists of 2,045 packets taken over 255 s, while the laboratory system capture contains 2,268 packets taken over 272 s.

In the off mode comparison, the packet size similarity is 1, implying that both traces have exactly the same distribution of packet sizes. Byte Throughput and Packet Throughput similarity metrics very close to unity (0.98493 vs. 0.98495, respectively); the closeness of these similarities follows from the packet size similarity. There were no error packets present in either capture, leading to an Error Count similarity of unity; the Invalid CRC metric is 1 for the same reason. The Function Code Count metric is 0.99996, indicating that the same proportion of function codes is present in both captures; similarly, the Function Code Sequence metric of 0.99964 indicates that the function codes appear in the same order in both captures. The ID Sequence Metric of 0.99995 indicates that, true to the MODBUS specification, the slave system responds

to master system, and the master system sends requests after the slave responds.

Timing metrics are also presented in Table 1. The master–master inter-arrival metric is 0.94912, while the master–slave inter-arrival similarity was 0.91356, and the overall inter-arrival time metric stands at 0.9353—all better than 90 %. Table 2 shows metrics for the ground tank system in auto mode; the similarity metrics for the auto case closely match those of the off case.

Traffic comparison similarity metrics for the pipeline system were similar to the results for the ground tank. These results were omitted for brevity.

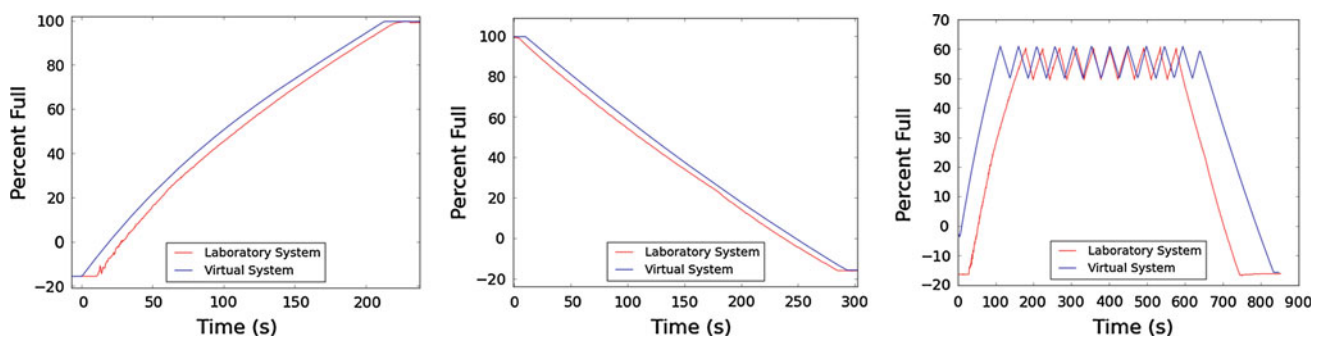
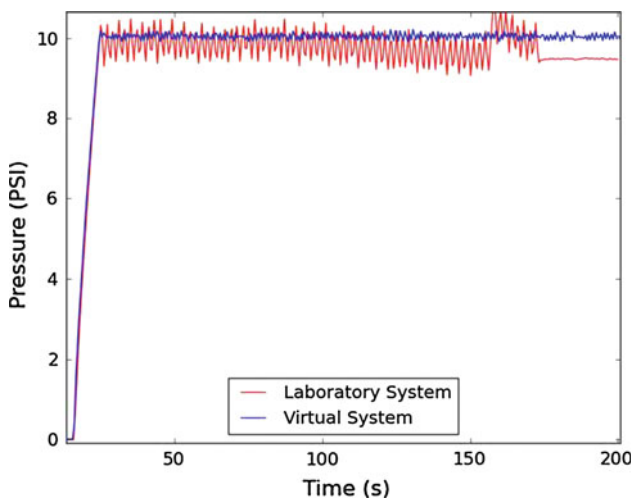
4.3 Process simulator versus actual process measurement comparison

Figure 5 provides a comparison of simulated ground tank water levels versus actual ground tank water levels measured in the laboratory. The left most graph in Fig. 5 shows the laboratory and virtual systems set in manual mode to increase from minimum to full level; this figure shows that the change in level in the two systems is nearly identical. The middle graph in Fig. 5 shows the water level decreasing from full to empty; this figure also shows that the change in level in the two systems is nearly identical. The right most graph in Fig. 5 shows the two systems starting from empty, being placed into auto mode, and then being turned off after several pump cycles. In this example, the virtual system was allowed to run for more cycles than the laboratory system; however, the behavior of both systems is similar.

Figure 6 shows the pipeline system pressures in the virtual testbed and actual measurement from the laboratory gas pipeline system in auto mode. The graph shows that a greater pressure variation is seen around the setpoint in the laboratory than in the virtual system; this is due to a mechanical switching delay in the valve solenoid that is not modeled in

Table 2 Similarity metrics for the ground tank system (auto mode)

Name	Similarity	Average error	Max. error	Percent error (%)
Byte throughput	0.98057	5.54963	5.54963	−3.812
Error count	1.00000	—	—	—
Function code count	0.99997	0.00003	0.00024	0.000
Function code sequence	0.99966	—	—	—
ID sequence	0.99998	—	—	—
Inter-arrival Time	0.93928	0.01099	0.07736	3.940
Invalid CRC	1.00000	—	—	—
Master-to-master inter-arrival time	0.94727	0.02302	0.08996	3.904
Master-to-slave inter-arrival time	0.92514	0.00488	0.06827	11.516
Packet size	0.99981	0.00513	8.50000	−0.027
Packet throughput	0.98070	0.31500	0.31500	−3.786

**Fig. 5** Ground tank simulation versus actual process measurement**Fig. 6** Ground tank simulation versus actual process measurement

the simulation. In spite of this difference, the behavior of the two systems is largely the same.

4.4 Integration with actual devices

This section details how the virtual testbed has been integrated with industrial control system equipment and com-

munications devices. The virtual testbed devices have been connected to each other through actual serial ports with commercial industrial control system radio equipment between the virtual master and slave. Using these radios, virtual masters have been paired with laboratory slaves, and laboratory masters have controlled virtual slave devices.

This demonstrates interoperability with industrial control system communications equipment. The radios used are 900 MHz proprietary industrial radios. The radios were connected to a USB serial port device to the virtual testbed host machine. Each of the virtual devices in both virtual testbeds were connected to individual PortLogger instances; each instance was connected to one serial port and created a pseudo-terminal that a virtual device connected to. The Port-Logger was acting as a virtual “bump-in-the-wire” logger for both physical serial ports. Laboratory system traffic was taken using serial tap cables placed between each PLC and radio, and only received packets were logged.

Similarity metrics were calculated comparing the virtual and laboratory ground tank systems in off and auto mode with the integrated industrial radio link. The auto mode laboratory capture contains 2,248 packets sent over 599 s, while the auto mode virtual capture contains 2,195 packets sent over 1,081 s. The off mode laboratory capture contains 2,789 packets sent

in 715 s, while the virtual capture of that system in off mode contains 2,142 packets sent over 1,052 s.

In off mode, the actual system had one packet with a CRC error, but the virtual system did not; this leads to a similarity score of 0. All other non-timing behavioral metrics, including Error Count, Function Code Count, Function Code Sequence, ID Sequence, and packet size, are still very high after radio integration. The timing metrics, which range from .68 to .78 in the two comparisons, are not as high as the other behavioral metrics. This is due to two reasons. First, the laboratory system used non-latency inducing taps for logging, while the simulated system used two PortLoggers, which act as store-forward loggers. The method of logging skewed the timing results of the simulated system. Second, the virtual system was connected using a USB to Serial adapter for two ports; virtual system packets must travel through 2 serial UART buffers, not just one. This added transmission time delay. In spite of the low similarities in timing (which may be correctable in future work), the results show that all non-timing-based behaviors of the virtual system were similar to the laboratory system. The significance of this conclusion for an industrial control system researcher is that in this case, implementing or testing projects (such as intrusion detection systems) that depend heavily on timing characteristics with this setup may lead to erroneous behavior or inconclusive results. However, results from projects not relying on timing can be trusted.

The virtual masters were used to control actual slave devices in the laboratory, and the virtual slaves were connected to actual masters from the laboratory. The virtual masters were able to monitor and control the laboratory slave devices connected to the actual physical process. Also, the real masters were able to monitor and control the virtual slave devices connected to the virtual physical process. All system operation modes were tested and were functional, and there were no error communications errors present in the system.

4.5 Virtual attacks versus actual attacks

This section detail attacks performed against the testbed to verify that the testbed can be used for testing attacks, not only generating “normal” industrial control system traffic. This quality is essential if the virtual testbed is to be used for security research. Three attacks are presented.

The first attack assumes that an insider or other attacker with physical access has placed a device on the serial line between the master and slave device in the ground tank system; this device can monitor communications and inject commands and responses. For this test, the attacking device injects a predetermined response to the master read request once every second; this response states that the tank level is 22.3 %. This attack is termed a “One Hertz Injection Attack”, and this attack was performed on both the laboratory and the virtual systems. The attack was effective

against both systems; however, the attack was more effective against the virtual system than the laboratory system. The time of sending of the attack packet is random with respect to the request/response “rhythm” of the system; the attack succeeds when a master sends a read request, and the first packet it receives is the injected packet. In the laboratory system, any data sent to the PLC before the request is sent are ignored. In the virtual system, the data are buffered until read by the industrial control system client code. Because there is a greater threshold of time for the attack to arrive to be successful against the virtual system than the laboratory system, the attack is more effective against the virtual system. Achieving complete fidelity for this attack would likely require adding a flush() call before sending a request in the industrial control system client code; this will be explored in future work. In any case, such behavior may be expected for two different industrial control system devices under attack. For example, there is no guarantee that a Siemens PLC and a Rockwell PLC will be vulnerable to this attack in exactly the same way. Because industrial control systems consist of heterogeneous components, the differing behavior does not negate the usefulness of the testbed.

The remaining attacks make use of the laboratory and virtual pipeline systems connected with the proprietary wireless system. These attacks assume that an attacker has infiltrated the radio system. One slave radio was attached to a PC to run the attacks from. Two radios were connected to a USB serial port device to the virtual testbed host machine. Each of the virtual devices in the virtual testbed was connected to individual PortLogger instances; each instance was connected to one serial port and created a pseudo-terminal that a virtual device was connected to. Laboratory system traffic was taken using serial tap cables placed between each PLC and radio, and only received packets were logged. The second attack demonstrated in the system is a slave denial of service attack against the pipeline system using the industrial control system radios. The attacker, with his own slave radio, is continuously transmitting data to cause the legitimate slave’s packets to not be received. The end result is that a master will not have updates for the responses sent by the slave and will store the same values for as long as the attack lasts. This attack was effective against both laboratory and virtual testbeds. In the third attack, the attacker transmits meaningless data consistently to create a denial of service to the slave while simultaneously responding to read requests. This creates only one response for the master to choose from, and it guarantees that the master receives only the values the attacker chooses. Once the attack is stopped, the pressure returns to the correct value within a few seconds. This attack was effective against both laboratory and virtual testbeds.

The similar behavior of the two radio attacks in both systems shows that the virtual testbed can be used to test industrial control system equipment. Although in some cases

attacks may seem to be more effective against the virtual systems, they still can be used to develop proof-of-concept attacks against industrial control system systems.

5 Conclusions and future work

This paper documents virtual testbed framework created using Python to provide independent industrial control system virtual devices, simulators, and logging devices. The virtual devices are able to control simulated processes, as well as communicate with each other, the simulator, and with actual industrial control system devices. Virtual devices are capable of supporting many more protocols than those implemented, which include MODBUS/TCP and MODBUS/RTU. Simulators approximate simulated processes based on control inputs from the virtual devices. Logging devices were developed to create faithful captures of virtual system traffic and also emulate the transmission characteristics of the medium.

Two virtual testbeds emulating laboratory-scale industrial control systems were developed, and traffic and behavior of these testbeds were compared to their laboratory counterparts. Virtual testbed behavior was verified quantitatively and by interoperability testing with laboratory equipment. Of the two, interoperability testing is more important because the end goal is a testbed that provides industrial control system functionality to researchers. However, quantitative measures that verify virtual testbed similarities provide assurances to researchers about which system features are nearly identical or indistinguishable from a real system, or those which may have some variance from an actual system. Where variances are present, researchers will know to exercise caution when developing and testing solutions that rely on the varying features. While we can show interoperability and statistical measures of system similarity, there is no existing work that sets a benchmark for how similar the system must be to be useful. For our initial implementation of testbeds, we have aimed for 84 % or better similarity metrics in network traffic, perfect interoperability with real equipment, and similar behavior under attack. While 84 % was selected arbitrarily, only future work that uses the testbed will be able to determine whether this level of similarity is sufficient.

The testbed masters and slaves were found to be interoperable with real industrial control system communications equipment and real devices. The virtual and laboratory testbeds were subjected to three attacks, and the two testbeds exhibited similar, though not exact behavior. Quantitatively, the first testbed (the ground tank) proved to have greater than 84 % similarity in the discussed similarity metrics in the two operation modes tested. The second testbed, the pipeline, showed greater than 84 % similarity in all but two defined metrics in both operating modes; two of these metrics will likely be improved in future work.

There are several natural extensions of this work. First, a repository for storing captures is necessary to organize contributed captures from laboratory and virtual testbeds. The repository should hold both the capture and searchable metadata, including descriptions of the captured activity and source attribution. Additionally, access to attack captures should be limited to trusted and vetted researchers. Second, several researchers have outlined attack taxonomies against common industrial control system protocols, including MODBUS and DNP3 [15–18]. Attacks described in these taxonomies should be run against the virtual systems to generate attack captures for the repository. Third, more testbed systems should be developed to exercise the flexibility in the testbed framework. Testbeds should be developed that provide more protocol diversity, integrate with higher-fidelity commercial simulation systems, and model much larger systems. Larger systems may be verified by partnering with industry. Fourth, a hardware interface to the simulator would allow the use of device discrete analog and digital inputs and outputs for pairing actual industrial control system devices with a simulated process. Such an interface will likely take the form of a microcontroller that provides discrete analog and digital input and outputs to the device. Input values would be read and sent to the simulator over a serial port or other communication scheme, while outputs from the microcontroller would be set by the values provided by the simulator. Finally, the unique paradigm of ladder logic in PLC programming makes translating ladder logic into traditional programming languages like C or Python difficult. A library- or domain-specific language for describing ladder logic in common high-level languages like C, Java, or Python would aid in developing new testbed device programming.

Acknowledgments This material is based upon work supported by the National Science Foundation Graduate Research Fellowship under Grant No. DEG-1125191.

References

1. Brugger, S.T.: KDD Cup '99 dataset considered harmful. <http://www.bruggerink.com/zow/GradSchool/KDDCup99Harmful.html>
2. Das, K.: Attack Development for Intrusion Detection Evaluation. Bachelor, MIT (2000)
3. Kendall, K.: A Database of Computer Attacks for the Evaluation of Intrusion Detection Systems. Master, MIT (1999)
4. Kennedy, T., Hunt, R.: A review of WPAN security: attacks and prevention. The International Conference on Mobile Technology, Applications & Systems, Ilan, Taiwan (2008)
5. KDD Cup 1999 Data. <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
6. McHugh, J.: Testing intrusion detection system: a critique of the 1998 and 1999 DARPA intrusion detection system evaluations as performed by Lincoln Laboratory. *ACM Trans. Inf. Syst. Secur.* **3**(4), 262–294 (2000)
7. Giani, A., Karsai, G., Roosta, T., Shah, A., Sinopoli, B., Wiley, J.: A testbed for secure and robust SCADA systems. 14th IEEE

- Real-time and Embedded Technology and Applications Symposium (RTAS'08) WIP session (2008)
8. Bergman, D.C.: Power grid simulation, evaluation, and test framework. Master's thesis, University of Illinois, Urbana-Champaign, IL (2010)
 9. Fovino, I., Masera, M., Guidi, L., Carpi, G.: An experimental platform for assessing SCADA vulnerabilities and countermeasures in power plants. *Human System Interactions (HSI)*, 2010 3rd Conference on, pp. 679–686 (2010)
 10. Morris, T., Srivastava, A., Reaves, B., Gao, W., Pavurapu, K., Reddi, R.: A control system testbed to validate critical infrastructure protection concepts. *Int. J. Crit. Infrastruct. Prot.* Elsevier. (2011). doi:[10.1016/j.ijcip.2011.06.00](https://doi.org/10.1016/j.ijcip.2011.06.00)
 11. Reddi, R., Srivastava, A.: Real time test bed development for power system operation, control and cyber security. *N. Am. Power Symp. (NAPS)* **2010**, 1–6 (2010)
 12. Brugger, S.T.: The Quantitative Comparison of Computer Networks. Doctoral dissertation, University of California, Davis, Davis, CA (2009)
 13. Cheung, S., Dutertre, B., Fong, M., Lindqvist, U., Skinner, K., Valdes, A.: Using model-based intrusion detection for SCADA networks. *Proceedings of the SCADA Security Scientific Symposium*. Miami, FL, Digital Bond (2007)
 14. Valdes, A., Cheung, S.: Communication pattern anomaly detection in process control systems. *2009 IEEE Conference on Technologies for Homeland Security*, Waltham, MA, USA, pp. 22–29 (2009)
 15. East, S., Butts, J., Papa, M., Sheno, S.: A taxonomy of attacks on the DNP3 protocol. *Crit. Infrastruct. Prot.* **III** **311**, 67 (2009)
 16. Fleury, T., Khurana, H., Welch, V.: Towards a taxonomy of attacks against energy control systems. In: Papa, M., Sheno, S. (eds.) *Critical Infrastructure Protection II*, IFIP International Federation for Information Processing. Springer, Boston, vol. 290, pp. 71–85 (2009). doi:[10.1007/978-0-387-88523-0_6](https://doi.org/10.1007/978-0-387-88523-0_6)
 17. Huitsing, P., Chandia, R., Papa, M., Sheno, S.: Attack taxonomies for the modbus protocols. *Int. J. Crit. Infrastruct. Prot.* **I**, 37–44 (2008)
 18. Iguere, V.: Security Assessment of SCADA Protocols: A Taxonomy Based Methodology for the Identification of Security Vulnerabilities in SCADA Protocols. VDM Verlag Dr. Muller, Saarbrücken (2008)